Numerical Algorithms

Intro and chapter 1

Chris Stolk

Contents of the course

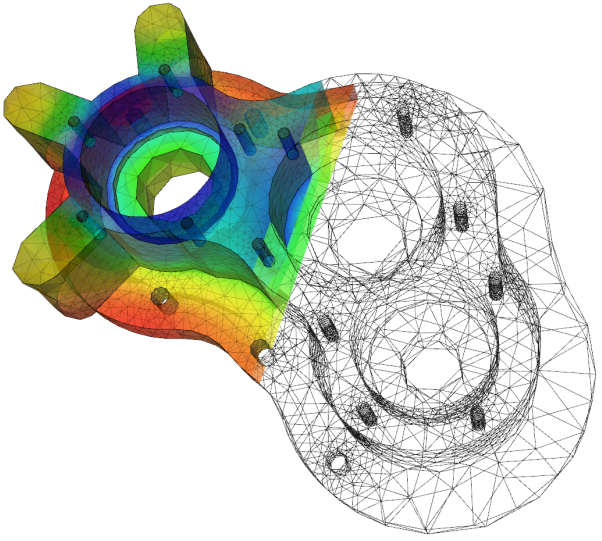
Course topics

- **Numerical linear algebra**: linear systems, linear least squares problems
- **Root-finding**, **optimization**
- **Numerical analysis**: interpolation, numerical integration, solving ordinary differential equations (ODEs)

Many applications:

- Simulating time-evolution of physical or biological systems
  $\rightarrow$ ODEs, linear systems, root-finding
- Computing electromagnetic fields in engineering
  $\rightarrow$ interpolation, numerical integration, linear systems
- Machine learning and data science
  $\rightarrow$ optimization, linear least squares

Application: heat simulation



Simulation of the temperature field in a pump casing by finite elements, using e.g. linear systems from this course. Methods for partial differential equations (PDEs), such as the heat equation, are treated in the elective course Scientific Computing (block 4).

Image: A1, Wikimedia Commons, CC BY-SA 3.0, computed with the open-source FEM software Elmer.

Organization

Lectures and **laptop classes** on Mondays and Thursdays (datanose)

3 TAs: Francesca Perlini, Giulia Pirovano, Ben Vigos

Book: Michael T. Heath, *Scientific Computing: An Introductory Survey*, revised 2nd edition

Other reading: slides of Heath (Canvas)

Assessment

- 1 midterm digital test 27 %
  Wed Nov. 25, location off Science Park, resembles a homework (Canvas)
- homeworks 13 %
  - 3 before the midterm test, one after
  - in groups of two, Python, Jupyter notebooks
  - I may replace the last homework by a compulsory exercise class
- Final exam 60 %
- Retakes for final exam and digital test

Today

- Computer arithmetic
  - Floating-point numbers
  - Operations with floating-point numbers
  - Pitfalls and fixes

  Why? Done the wrong way, a computation can give large errors. These pitfalls are not difficult to avoid, but you have to be aware of them.
- Approximation in scientific computing
  - Types of errors
  - Condition numbers
  - Examples

How a computer stores a number

Floating-point system: base β, precision p

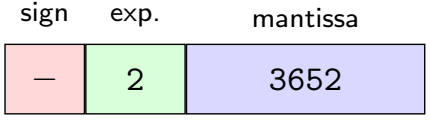
$$x = \pm \left(d_0.d_1 d_2 \cdots d_{p-1}\right)_\beta \times \beta^E$$

Steps to store a real number x:

1. Normalize: write $x = \pm m \times \beta^E$ with $1 \leq m < \beta$
2. Round m to p digits
3. Store the sign
4. Store the exponent E
5. Store the mantissa $d_0 d_1 \cdots d_{p-1}$

Example: store $x = -365.2422$ with $p = 4$ digits, decimal system ($\beta = 10$)

1. $x = -3.652422 \times 10^2$
2. $\mathrm{fl}(x) = -3.652 \times 10^2$
3. sign: $-$
4. exponent: 2
5. mantissa: 3652



Storing a number: points to note

Exponent range

- The exponent must satisfy $L \leq E \leq U$
- If $E > U$: overflow

Rounding

- Round to nearest
- Tie: round to even ($p = 4$: $1.2345 \to 1.234$, $1.2355 \to 1.236$)

Question: why round to even in case of a tie?

Subnormal numbers

- If $E < L$: write x with exponent L
- Store the mantissa with leading zeros

Example: $p = 4$, $\beta = 10$, $L = -99$

$$\begin{aligned} x &= 1.234 \times 10^{-101} \\ &= 0.01234 \times 10^{-99} \\ \mathrm{fl}(x) &= 0.012 \times 10^{-99} \end{aligned}$$

mantissa: 0012 (fewer significant digits)

Machine precision

Heath's definition: ϵ_{mach} = maximum relative rounding error

$$\frac{|\text{fl}(x) - x|}{|x|} \leq \epsilon_{\text{mach}}$$

For decimal FP (rounding to nearest):

$$\epsilon_{\text{mach}} = 0.5 \cdot 10^{-p+1}$$

Note: Heath uses a different convention than is usual in computer science (C, Python):

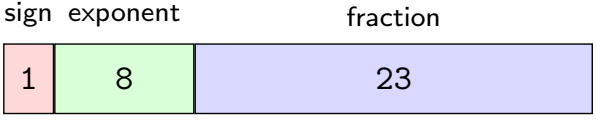
$$\epsilon_{\text{Heath}} = 0.5 \cdot \epsilon_{\text{C, Python}}$$

Proof for $p = 3$: numbers between 1 and 9.99

Standard formats: single and double precision

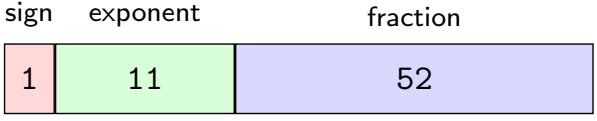
- Computers use binary FP ($\beta = 2$): digits are 0's and 1's
- Machine precision: $\epsilon_{\text{mach}} = \frac{1}{2} \cdot 2^{-p+1} = 2^{-p}$

Single precision (32 bits)



- $p = 24$, $L = -126$, $U = 127$
- min. norm.: $2^{-126} \approx 1.2 \cdot 10^{-38}$
- max.: $\approx 2^{128} \approx 3.4 \cdot 10^{38}$
- $\epsilon_{\text{mach}} = 2^{-24} \approx 6.0 \cdot 10^{-8}$

Double precision (64 bits)



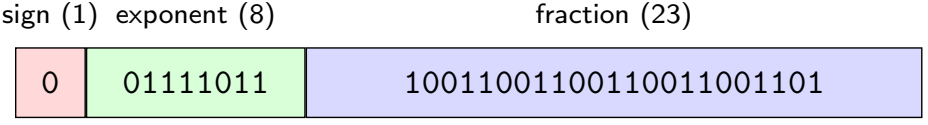
- $p = 53$, $L = -1022$, $U = 1023$
- min. norm.: $2^{-1022} \approx 2.2 \cdot 10^{-308}$
- max.: $\approx 2^{1024} \approx 1.8 \cdot 10^{308}$
- $\epsilon_{\text{mach}} = 2^{-53} \approx 1.1 \cdot 10^{-16}$

Special values: `Inf`, `-Inf` (e.g. 1/0, overflow), `NaN` = not-a-number (e.g. 0/0)

Example in binary: $x = 0.1$ in IEEE single precision

$\beta = 2$, $p = 24$, 32 bits in total

1. Normalize: $0.1 = (1.10011001100110011001100\ldots)_2 \times 2^{-4}$
2. Round to 24 bits: $\mathrm{fl}(0.1) = (1.\underbrace{10011001100110011001101}_{23\text{ bits}})_2 \times 2^{-4}$
3. Sign: $+$ $\quad \rightarrow$ bit 0
4. Exponent: $E = -4$ $\quad \rightarrow$ stored as $E + 127 = 123 = (01111011)_2$
5. Mantissa: leading bit is always 1, so only the 23 bits of the fraction are stored



Stored value: $\mathrm{fl}(0.1) = 0.100000001490116\ldots \neq 0.1$

Arithmetic operations

Let $\oplus$ denote computer addition of machine numbers. Then

$$x \oplus y = \text{round}(x + y) \quad \text{(to } p \text{ digits)}$$

Hence

$$x \oplus y = (x + y)(1 + \delta), \qquad |\delta| < \epsilon_{\text{mach}}$$

Same for subtraction, multiplication, division

Pitfalls

- Adding small x to large y: lose precision of x
- Subtracting nearly equal numbers (*cancellation errors*)
- (and also overflow/underflow)

Pitfall: cancellation errors

Example

$$\begin{array}{rrl} \hat{a} = & 123.456 & \\ \hat{b} = & 123.455 & \text{6 digits} \\ \hline \hat{a} - \hat{b} = & 0.001 & \text{1 digit} \end{array}$$

This could come from true values

$$\begin{array}{rr} a = & 123.4556 \\ b = & 123.4554 \\ \hline a - b = & 0.0002 \end{array}$$

true value	0.0002
approx. value	0.001

$\Rightarrow$ 400 % error

Avoid subtracting nearly equal numbers

Example

Compute the positive root of

$$ax^2 + x - 1 = 0$$

for $a = 1, 10^{-2}, 10^{-4}, \ldots, 10^{-18}$:

$$x = \frac{-1 + \sqrt{1 + 4a}}{2a}$$

Obvious questions

- What causes the errors?
- Can we fix it?

a	$x(a)$	$x_{\mathrm{ref}}(a)$
1e+00	0.61803399	0.61803399
1e-02	0.99019514	0.99019514
1e-04	0.99990002	0.99990002
1e-06	0.99999900	0.99999900
1e-08	0.99999999	0.99999999
1e-10	1.00000008	1.00000000
1e-12	0.99997788	1.00000000
1e-14	0.99920072	1.00000000
1e-16	1.11022302	1.00000000
1e-18	0.00000000	1.00000000

Double precision; reference values computed with 80 digits

Example revisited

Positive root of $ax^2 + x - 1 = 0$

abc formula:

$$x = \frac{-1 + \sqrt{1 + 4a}}{2a}$$

Example revisited

Positive root of $ax^2 + x - 1 = 0$

abc formula:

$$x = \frac{-1 + \sqrt{1 + 4a}}{2a}$$

New formula:

$$x = \frac{2}{1 + \sqrt{1 + 4a}}$$

Relative errors of order 10^{-16}: the best one can expect

(Cf. book exercise 1.15)

a	$x(a)$ (new formula)	rel. error
1e+00	0.6180339887498948	-9e-17
1e-02	0.9901951359278481	-2e-16
1e-04	0.9999000199950013	-7e-17
1e-06	0.9999990000019999	-7e-17
1e-08	0.9999999900000002	-3e-17
1e-10	0.9999999999000000	-8e-18
1e-12	0.9999999999989999	-9e-17
1e-14	0.9999999999999900	8e-18
1e-16	1.0000000000000000	1e-16
1e-18	1.0000000000000000	1e-18

Overflow

Double precision FP numbers have a max. of $\approx 10^{308}$

Example: norm of a vector $(x_1, \ldots, x_n)$

$$\|x\| = \sqrt{x_1^2 + \cdots + x_n^2}$$

The sum of squares can overflow, while $\|x\|$ itself may be okay.

Solution:

Today

- Computer arithmetic
  - Floating-point numbers
  - Operations with floating-point numbers
  - Pitfalls and fixes
- Approximation in scientific computing
  - Types of errors
  - Condition numbers
  - Examples

  Why? Small input errors can give large output errors, in the problem itself and in each step of an algorithm.

Absolute and relative errors

Let x = true value, $\hat{x}$ = approx. value

(absolute) error $= \Delta x = \hat{x} - x$

relative error $= \dfrac{\Delta x}{x} = \dfrac{\hat{x} - x}{x}$

Note: the error can be positive or negative. Sometimes we just care about $|\Delta x|$ or $|\Delta x / x| \Rightarrow$ mention explicitly.

Computational and input error

Let also f = function/operation to compute, $\hat{f}$ = approx. of f,
x = exact input, $\hat{x}$ = approx. input

Error in $f(x)$:

$$\hat{f}(\hat{x}) - f(x) = \underbrace{\hat{f}(\hat{x}) - f(\hat{x})}_{\text{computational error}} + \underbrace{f(\hat{x}) - f(x)}_{\text{propagated input error}}$$

Rounding and truncation error

Two kinds of computational error:

Rounding error $\rightarrow$ saw this in computer arithmetic

Truncation error $\rightarrow$ error due to other approximations

Example:

$$y = \exp(x) = 1 + x + \tfrac{1}{2}x^2 + \tfrac{1}{6}x^3 + \cdots$$

Approximation $\hat{y} = 1 + x$

Then $y - \hat{y} = \tfrac{1}{2}x^2 + \tfrac{1}{6}x^3 + \cdots$ is the truncation error

Forward and backward error

Assume we have

- true input x
- true function $f : x \mapsto y$
- approximation $\hat{y}$ to y

Then

$$\text{forward error} = \hat{y} - y$$

Let $\hat{x}$ be such that $f(\hat{x}) = \hat{y}$. Then

$$\text{backward error} = \hat{x} - x$$

Backward error is the error in x that corresponds to the error in y by error propagation.

(another example: Heath's slides, p. 20)

Example: forward and backward error

Approximate $y = f(x) = \cos(x)$ for $x = 1$ by truncating the series

$$\cos(x) = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \cdots$$

after two terms:

$$\hat{y} = 1 - x^2/2 = 0.5$$

What are the forward and backward errors?

(Heath, Example 1.6)

True value: $y = \cos(1) \approx 0.5403$

Forward error:
$\hat{y} - y \approx 0.5 - 0.5403 = -0.0403$

Find $\hat{x}$ such that $\cos(\hat{x}) = \hat{y}$:

$$\hat{x} = \arccos(0.5) \approx 1.0472$$

Backward error: $\hat{x} - x \approx 0.0472$

Condition number

About propagated input error

$$\begin{aligned}\text{cond} &= \max \frac{|\text{rel. forward error}|}{|\text{rel. backward error}|} \\ &= \max \left| \frac{\Delta y / y}{\Delta x / x} \right|\end{aligned}$$

max over different input values x and input errors Δx (worst case)

- "max error multiplication factor"
- independent of numerical method
- large cond $\rightarrow$ problem is called *sensitive* (or ill-conditioned)

(Heath's slides, pp. 26–33)

Evaluating a function

Suppose we compute $y = f(x)$, x given. Approximate $\mathrm{cond}(f)$:

$$\mathrm{cond} = \left| \frac{\Delta y / y}{\Delta x / x} \right|$$

- x = true input, $y = f(x)$ = true output
- $\hat{y}$ = approx. output
- $\hat{x}$ is such that $f(\hat{x}) = \hat{y}$ (Δx is the backward error)

In general $|\hat{x} - x|$ is small. By Taylor expansion

$$\hat{y} = f(\hat{x}) \approx f(x) + (\hat{x} - x) f'(x)$$

so $\hat{y} - y \approx (\hat{x} - x) f'(x)$ and

$$\mathrm{cond}(f) \approx \left| \frac{(\hat{x} - x) f'(x) / f(x)}{(\hat{x} - x)/x} \right|$$

$$\boxed{\mathrm{cond}(f) = \left| \frac{x f'(x)}{f(x)} \right|}$$

Example: numerical differentiation

The derivative can be approximated by

$$f'(x) \approx \frac{f(x+h) - f(x)}{h} =: \hat{y} \qquad (*)$$

Plan: understand the errors (two types), and the errors vs. h

Truncation error (since $(*)$ is not exact)

Taylor expansion:
$f(x+h) = f(x) + hf'(x) + \frac{1}{2}h^2 f''(\theta)$, θ between x and $x+h$. So

$$\frac{f(x+h) - f(x)}{h} = f'(x) + \underbrace{\frac{1}{2}hf''(\theta)}_{\text{truncation error}}$$

Rounding error

Instead of $(*)$ we compute $\frac{\hat{f}(x+h) - \hat{f}(x)}{h}$, with $\hat{f}(x)$ the floating-point approximation of $f(x)$: $|\hat{f}(x) - f(x)| \approx \epsilon_{\text{mach}}|f(x)|$.

$$\text{rounding error} \approx \frac{2\epsilon_{\text{mach}}}{h}|f(x)|$$

Truncation error grows with h, rounding error grows with $1/h$ (cancellation errors!)

Numerical differentiation: total error

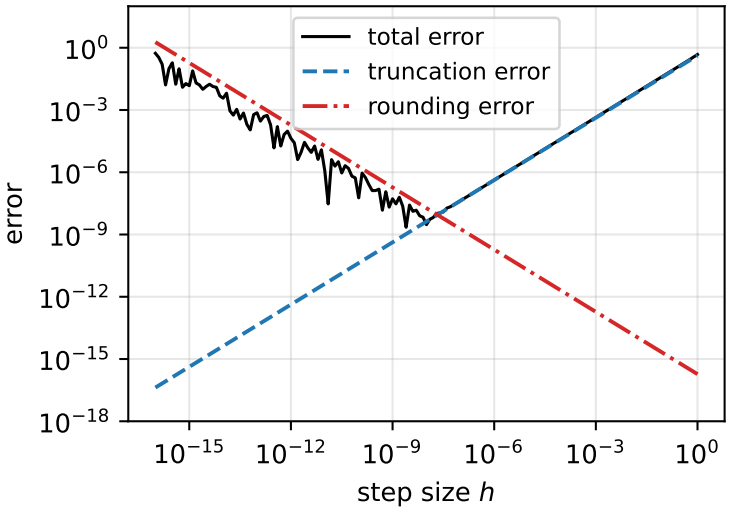
Example: $f(x) = \sin(x)$, $x = 1$,
true $f'(x) = \cos(x)$

Total error

$$\approx C_1 h + C_2 \frac{\epsilon_{\text{mach}}}{h}$$

Remarks

- Optimal error is $O(\sqrt{\epsilon_{\text{mach}}})$, for $h \sim \sqrt{\epsilon_{\text{mach}}}$



Condition number of solving an equation

Problem: given f, find x such that $f(x) = 0$

This is the inverse of evaluating $y = f(x)$: given $y = 0$, find x.

In practice we have an approximation $\hat{f}$ of f, e.g. due to rounding errors:

$$|\hat{f} - f| \leq \delta \quad \text{(near the root)}$$

The computed root $\hat{x}$ satisfies $\hat{f}(\hat{x}) = 0$, so $|f(\hat{x})| \leq \delta$.

Taylor expansion around the root x:

$$f(\hat{x}) \approx f(x) + (\hat{x} - x)f'(x) = (\hat{x} - x)f'(x)$$

so

$$|\hat{x} - x| \approx \frac{|f(\hat{x})|}{|f'(x)|} \leq \frac{1}{|f'(x)|}\,\delta$$

- Absolute condition number: $1/|f'(x)|$
- Absolute, because $f(x) = 0$: a relative error in f makes no sense
- Small $|f'(x)| \to$ sensitive problem

Example: solving an equation

$$f(x) = (x - 10)(x - 10.1)(x - 10.2) = x^3 - 30.3x^2 + 306.02x - 1030.2$$

Suppose we numerically solve for the root 10.1 using the second form of f. Estimate the rounding errors in $f(x)$ for x around 10.1, and then estimate the error in the root.

Example: solving an equation

$$f(x) = (x - 10)(x - 10.1)(x - 10.2) = x^3 - 30.3x^2 + 306.02x - 1030.2$$

Suppose we numerically solve for the root 10.1 using the second form of f. Estimate the rounding errors in $f(x)$ for x around 10.1, and then estimate the error in the root.

- Terms of size $\approx 3 \cdot 10^3$ are added $\rightarrow$ cancellation: a few rounding errors of size $3 \cdot 10^3 \, \epsilon_{\text{mach}} \approx 3 \cdot 10^{-13}$, so $\delta \approx 10^{-12}$
- $f'(10.1) = (0.1)(-0.1) = -0.01$, cond $= 100$
- Expected error in the root $\approx 100 \, \delta \approx 10^{-10}$

Illustration (Python, double precision):

x	rounding error in $f(x)$
$10.1 - 4 \cdot 10^{-11}$	$-6.3 \cdot 10^{-13}$
$10.1 - 2 \cdot 10^{-11}$	$2.7 \cdot 10^{-14}$
10.1	$-2.3 \cdot 10^{-13}$
$10.1 + 2 \cdot 10^{-11}$	$-4.8 \cdot 10^{-13}$
$10.1 + 4 \cdot 10^{-11}$	$1.7 \cdot 10^{-13}$

Root found by `scipy.optimize.brentq`:
`10.099999999931079`, error $-6.9 \cdot 10^{-11}$

What is needed for good computational results?

Problem side

Problem well posed

- has a solution
- solution is unique
- depends continuously on input

Problem well-conditioned

- does not blow up input errors too much

Method side

Method is stable

- errors incurred during computation are not "blown up"